

SOFTWARE DEVELOPMENT METHODOLOGIES

1. Waterfall Method

Waterfall is the classic and most traditional paradigm in software development, first presented by Winston W. Royce in his 1970 paper, **"Managing the Development of Large Software Systems."** Its core concept is a **rigid, linear, and sequential flow**, designed to bring structure to the development process. The entire project is divided into a series of distinct, consecutive phases: Requirements Gathering and Analysis, System Design, Implementation (Coding), Testing, Deployment, and Maintenance. Its key principle is a strict, one-way progression; a phase cannot begin until the previous one is fully completed and signed off. The model is **explicitly not iterative** and does not define a process to go back to a previous phase. This makes it most suitable for smaller projects where requirements are completely understood and stable from the very beginning.

2. Spiral Method

First introduced by Barry Boehm in 1986 and refined in his 1988 paper, **"A Spiral Model of Software Development and Enhancement,"** the Spiral model was specifically designed to address the limitations of the Waterfall model for large, complex, and high-risk projects. Its paradigm is a **purely risk-driven framework** that uniquely combines the systematic approach of Waterfall with the iterative refinement of Prototyping. The core concept is that the entire development process is a spiral, where each loop represents a full project cycle. This model is **inherently iterative**, where the iteration is the entire process loop itself. Each loop consists of four phases: (1) Planning, (2) **Risk Analysis** (the model's most critical phase), (3) Engineering (which often includes prototyping), and (4) Customer Evaluation. The project starts on a small scale and grows with each loop of the spiral, with risks being identified and mitigated at every turn.

3. RAD

Emerging in the 1980s and formalized by James Martin in his 1991 book, **"Rapid Application Development,"** RAD is a paradigm built as a direct response to the slow and inflexible nature of the Waterfall model. Its main objective is to **accelerate the entire software development process**. The core concept is to prioritize iterative development, active user involvement, and rapid prototyping over lengthy upfront planning. The paradigm follows four main phases: Requirements Planning, User Design, Construction, and Cutover. Its iteration is a **rapid-fire, informal cycle concentrated specifically between the User Design and Construction phases**. In this tight loop, developers and users work together to design, build, and review system components. The use of **"time-boxing"** (fixing a non-negotiable timeframe) is key to maintaining speed.

4. Prototype Method

Prototype model is not a complete, end-to-end software development lifecycle (SDLC). Instead, it is a **specific technique** whose paradigm is centered on **clarifying unknown requirements through user feedback**. The term was formally introduced in a 1981 paper by Alan Borning in the journal "**ACM Transactions on Programming Languages and Systems.**" This technique is used when a client is unsure of what they want or when requirements are fuzzy. The core concept is to build an early version of the product—a mock-up or simulation with basic functionality—to give users something tangible to react to. The process is inherently iterative, following a clear cycle: (1) Gather initial requirements, (2) Perform a quick design, (3) Build the prototype, (4) Have the user evaluate it, and (5) Refine the prototype based on feedback. This **Build → Evaluate → Refine** loop continues until the requirements are fully understood.

5. Feature-Driven Development

Feature-Driven Development (FDD) was first devised in 1997 by Jeff De Luca to meet the needs of a 15-month, 50-person software project at a large Singapore bank. Its concepts were later introduced in *Java Modeling in Color with UML* (1999) by Peter Coad, Eric Lefebvre, and Jeff De Luca, and further expanded in Stephen Palmer and Mac Felsing's book *A Practical Guide to Feature-Driven Development* (2002). What makes FDD unique is its **five structured processes**: *Develop Overall Model, Build Feature List, Plan by Feature, Design by Feature, and Build by Feature*. The first two steps create a strong foundation, while the last three are repeated iteratively, combining clarity, structure, and agility to deliver reliable results.

6. Lean Software Development

Lean Software Development (LSD) traces its roots to the **Toyota Production System** of the 1980s, which focused on waste elimination, quality improvement, and continuous learning. The principles were popularized in the 1990s through *The Machine That Changed the World* and adapted into software by Mary and Tom Poppendieck in the early 2000s with their book *Lean Software Development: An Agile Toolkit*. By the 2010s, LSD had become widely integrated into Agile practices, and in the 2020s it continues to evolve in modern development. Its uniqueness lies in the application of **seven lean principles**—eliminating waste, building quality in, delivering fast, empowering the team, delaying decisions until facts are clear, optimizing the whole, and continuous improvement. It emphasizes value stream mapping and Kaizen, much like the Minimum Viable Product (MVP) approach, ensuring efficiency and real value delivery.

7. Dynamic Systems Development Model (DSDM)

The Dynamic Systems Development Model (DSDM) emerged in **1994** when the DSDM Consortium was founded to bring structure to the Rapid Application Development (RAD) approach, which was widely used in the early 1990s but criticized for quality issues. Over time,

DSDM was refined, releasing public versions in 2006, a handbook in 2014, and rebranding in 2016 under the Agile Business Consortium. DSDM is unique in that it **balances agility with strong governance**. Unlike most Agile approaches that fix scope and allow time and cost to vary, DSDM fixes **time, cost, and quality** from the start while allowing scope to remain flexible. This ensures discipline while still enabling adaptability.

8. Joint Application Development (JAD)

Joint Application Development (JAD) was developed in the **late 1970s** by Chris Morris and Tony Crawford with the goal of improving client satisfaction. Through the 1980s, it was validated via collaborative workshops and has since been applied successfully across industries to produce high-quality systems. The key uniqueness of JAD lies in its **active involvement of clients and end users** throughout the development process. Instead of only gathering requirements at the beginning, JAD integrates stakeholders into the **design and decision-making stages** through structured workshops called *JAD sessions*. This ensures better alignment between user needs and final product outcomes.

9. Agile Development Methodology

Historically, Agile emerged in the late 1990s and was formally established with the **Agile Manifesto in 2001**. It was created by a group of software developers as a direct response to the failures of traditional, rigid methodologies like the **Waterfall Model**. These older methods were proving too slow, inflexible, and risky for the fast-paced software industry, as they required all planning to be done upfront and did not easily accommodate changes.

The paradigm of Agile represents a fundamental shift from a linear, plan-driven process to a flexible, value-driven one. Instead of building a complete product in one long cycle, the Agile paradigm is **iterative and incremental**. Work is broken down into short cycles called **sprints**, with the goal of delivering a small, working piece of the product at the end of each cycle. This approach embraces change, prioritizes customer collaboration, and relies on continuous feedback to ensure the final product truly meets user needs.

10. Scrum Development Methodology

The history of Scrum predates the Agile Manifesto. It was first formulated by **Ken Schwaber** and **Jeff Sutherland** in the early 1990s. They drew inspiration from a 1986 Harvard Business Review article, "The New New Product Development Game," which described a faster, more flexible development approach used by companies in Japan, comparing it to the sport of rugby—hence the name **"Scrum."**

Schwaber and Sutherland's work was a direct reaction to the failures of the traditional **Waterfall model**. They based Scrum on **empirical process control theory**, which asserts that knowledge comes from experience and making decisions based on what is known. This was a radical departure from the plan-driven approaches of the time. Scrum was later one of the primary frameworks that influenced the creation of the **Agile Manifesto in 2001**, solidifying its place as a leading Agile methodology.

The paradigm is implemented through a simple set of roles, events, and artifacts that work together to bring this empirical process to life. In summary, the Scrum paradigm creates a rhythm of short, repeated cycles where a team can deliver a valuable piece of a product, get feedback, and adapt their plan, significantly reducing risk and increasing focus on customer value.

11. Extreme Programming Methodology

Extreme Programming was created by **Kent Beck** in the late 1990s while he was leading the **Chrysler Comprehensive Compensation System (C3)** project. The project was struggling, and Beck was brought in to turn it around. He began refining development practices he had been working on, emphasizing a disciplined and high-quality approach to agile software development.

The success of these refined practices on the C3 project led Beck to formalize them. He published his ideas in the influential 1999 book, "**Extreme Programming Explained: Embrace Change.**" XP became one of the most prominent and specific of the early Agile methodologies and was a key influence on the creation of the **Agile Manifesto in 2001**. Its history is rooted in solving real-world project problems by focusing intensely on engineering discipline.

The **Extreme Programming (XP)** paradigm is a software development approach centered on technical excellence and effective communication, designed to produce high-quality software while adapting to changing requirements. It is guided by five core values: **Simplicity, Communication, Feedback, Respect, and Courage.**

This paradigm is put into action through a set of disciplined engineering practices taken to "extreme" levels. The most notable practices include:

1. **Test-Driven Development (TDD)**
2. **Pair Programming**
3. **Continuous Integration (CI)**
4. **Simple Design**
5. **Having an On-Site Customer**

The goal is to create a disciplined, collaborative environment that allows teams to deliver high-quality software at a sustainable pace.

12. DevOps Methodology

The term **DevOps** was coined by Patrick Debois in 2009, inspired by a groundbreaking presentation a year earlier by two engineers from Flickr titled "10+ Deploys Per Day: Dev and Ops Cooperation at Flickr." At the time, even highly effective Agile teams faced a significant bottleneck: the "wall of confusion" between the **Development (Dev)** team, who wanted to release new features quickly, and the **Operations (Ops)** team, who prioritized stability and were resistant to change.

This conflict created long delays, blame, and inefficiency. The DevOps movement grew out of the Agile and IT operations communities as a direct response to this problem. It sought to extend the principles of Agile beyond just the development team to encompass the entire delivery lifecycle, from code creation all the way to production deployment and maintenance.

The DevOps paradigm is a cultural philosophy and set of practices that unifies software development (Dev) and IT operations (Ops) to deliver software faster and more reliably.

Comparison by industry company point of view based on:

1. Cost:

Lean is the most cost-effective by eliminating waste, followed by **JAD**, **Agile**, and **Scrum** which offer controlled costs through shorter analysis and iterative development. **RAD**, **XP**, and **Prototype** are in the moderate tier, but their costs can escalate with intensive user involvement, extensive testing, or frequent revisions. **FDD** and **DSDM** tend to be more expensive due to significant upfront investment. **Waterfall** is costly because of the high price of late-stage changes, and **Spiral** is the most expensive due to its mandatory, repeated risk analysis. **DevOps** has a high initial cost but becomes highly cost-effective in the long term.

2. Duration:

RAD is the fastest methodology, followed closely by **JAD**, **Scrum**, and **XP**, which accelerate projects through condensed design phases and short iterative cycles. **Lean**, **DSDM**, and **Prototype** are also rapid due to their focus on early delivery, though prototyping's duration can extend with revisions. **FDD** is moderately paced because of its required upfront modeling. **Waterfall** is inherently slow due to its rigid sequential nature, while **Spiral** is the slowest by design, prioritizing in-depth risk analysis over speed. **DevOps** has a unique timeline with a slow initial setup followed by an extremely rapid, continuous delivery pace.

3. Tools:

JAD, **Lean**, and **Waterfall** have the lowest tool dependency, relying on general-purpose software like MS Project or physical boards. **Agile** frameworks like **Scrum**, **XP**, and **FDD** have a medium dependency, benefiting greatly from project management tools like Jira, as does **Prototyping** with its reliance on design tools like Figma. The dependency becomes high with **RAD**, which needs CASE tools to achieve its speed, and the **Spiral** model, which often requires specialized risk modeling software. **DevOps** has a fundamental dependency, where the methodology is inseparable from its integrated toolchain for automation, CI/CD, and monitoring.

4. Human Resources

The choice of a development methodology fundamentally shapes a company's HR strategy. It dictates who you hire, how you structure teams, what culture you foster, and what skills you need to develop.

The software development methodologies can be classified into four main groups, each representing an evolution in thinking from rigid, plan-driven processes to flexible, value-driven ones.

- a. Traditional & Sequential Models: Waterfall Model, Spiral Mode
- b. Iterative & Collaborative Models: Prototype Methodology, RAD, JAD, DSDM
- c. Agile & Lean Models: Agile, Scrum, XP, FDD, Lean Development
- d. DevOps Methodology

HR Point of View Comparison can be presented in the following table

Methodology Category	Key HR Focus	Skills in Demand	Team Structure	Work Culture
Traditional	Predictability & Control	Deep Specialization	Hierarchical & Siloed	Plan-Driven
Iterative	Facilitation & Early Collaboration	Communication & Facilitation	Collaborative by Phase	Idea Validation
Agile & Lean	Team Empowerment	T-Shaped & Soft Skills	Cross-Functional & Self-Organizing	Adaptive & Transparent
DevOps	End-to-End Ownership	Automation & Hybrid Skills	Integrated Teams (Dev+Ops)	Ownership & Blameless

Based on the table:

From an HR perspective, the approach evolves from recruiting siloed **specialists** for **Traditional** models and facilitating **collaboration** in **Iterative** ones, to cultivating empowered, cross-functional **Agile** teams, and finally to building integrated **DevOps** teams with a culture of complete ownership.

5. Product

The choice of methodology has a direct and profound impact on the product itself—affecting its features, quality, speed of delivery, and ability to meet user needs.

Product Point of View Comparison can be presented in the following table

Methodology Category	Feature Development	Product Quality	Delivery Speed	Feedback & Adaptation
Traditional	Scope is defined and frozen upfront.	Checked at the end; bugs are expensive to fix.	Very slow; single "big bang" release.	Almost none; high risk of building the wrong product.
Iterative	Features are refined based on user prototypes.	High UX/design quality; technical quality addressed later.	Faster design, but final delivery can still be slow.	Excellent for design/UI, less for core functionality.
Agile & Lean	Highest value features are built first; scope is flexible.	Quality is built-in and is everyone's responsibility.	Fast and incremental; working software every few weeks.	Constant feedback loop ensures high product-market fit.
DevOps	Continuous updates; features can be released multiple times a day.	Focus on reliability and stability through automation and monitoring.	Fastest possible via automated CI/CD pipelines.	Real-time feedback from live monitoring.

Based on the table:

While **Traditional** methodologies execute a rigid, upfront plan, **Iterative** models refine that plan with early user feedback, **Agile** continuously adapts by delivering value in short cycles, and **DevOps** automates the entire delivery process for the fastest possible feedback and release.

Comparison base on stakeholder's perspective

1. Cost

Lean remains the most cost-efficient, followed by **Prototype** which is affordable for validating ideas. **Scrum** and **Agile** are moderate but can be efficient since third-party involvement starts early. **FDD** is relatively stable for large-scale projects, while **Waterfall** appears stable but becomes costly when revisions are required with external vendors. **XP** and **RAD** tend to be more expensive due to strict practices and intensive team needs. **DSDM** and **JAD** are higher in cost because of collaborative workshops, **DevOps** is costly due to shared infrastructure requirements, and **Spiral** is the most expensive since it involves extensive risk analysis with third parties.

2. Duration

RAD is the fastest, especially for collaborative prototyping, followed by **Scrum** and **Agile** which are agile in involving third parties. **Lean** is efficient in execution, **Prototype** is fairly quick for validation, and **FDD** is slower as it requires structured feature development. **XP** is moderate due to its technical discipline, while **DSDM** and **JAD** take longer because of multiple collaborative sessions. **DevOps** is moderate as automation requires setup, **Waterfall** is slow due to its linear nature with vendors, and **Spiral** is the slowest because of repeated risk evaluations.

3. Quality of Product

Agile delivers the highest quality as it is flexible and involves third parties in iterations, followed by **Scrum** with its frequent reviews. **JAD** also ranks high as it directly involves stakeholders, while **DevOps** ensures quality through continuous integration with external partners. **XP** provides strong technical quality, **Spiral** ensures quality through risk analysis, and **FDD** maintains consistency across features. **DSDM** offers decent quality with strict documentation, **Prototype** delivers moderate quality with simple iterations, **Waterfall** is standard but often mismatches third-party needs, **RAD** is fast but risks compromising quality, and **Lean** focuses on efficiency so some quality details may be overlooked.

Future of Software Development Methodologies

1. Cloud-native development:

Cloud-native development involves building applications specifically for cloud environments and leveraging cloud services and platforms to create highly scalable, resilient, and efficient applications. This approach is becoming increasingly important as more organizations move their IT infrastructure to the cloud. Cloud-native development allows us to take advantage of the benefits of cloud computing, such as elastic scalability, high availability, and reduced infrastructure costs. It also allows for easier deployment and management of applications, as well as faster time-to-market for new features and updates [\[1\]](#).

2. Low-Code/No-Code Development

Software engineering is moving toward simplification, with a new approach to creating software solutions known as low-code/no-code (LCNC) development. LCNC platforms enable practically anyone to develop software without traditional coding via graphical user interfaces and configuration. Such programming environments have great potential because they allow people to implement their ideas into digital solutions, even with limited or no software development skills.

- 80% of respondents believe that the low-code approach can free up developers so that they can focus on higher-level projects.
- 72% of low-code developers reportedly create software apps twice as fast as those using traditional development methods.
- The global value of the LCNC development platform market is projected to be \$46.4 billion by 2026, growing at a CAGR of 25% during this period.

Some developers are still skeptical about low-code/no-code development. But the fact is that this approach brings many advantages. By automating much of the process, low-code tools save time and money. Moreover, it has become easier for newcomers to enter the world of programming [\[2\]](#).

3. AI Driven Development Life Cycle (AI-DLC)

The AI-Driven Development Lifecycle (AI-DLC) is a transformative methodology that integrates artificial intelligence as a central collaborator in software development. Unlike traditional approaches that treat AI as a supplementary tool, AI-DLC positions AI to actively drive workflows while maintaining critical human oversight. This methodology aims to enhance productivity, accelerate development, and improve software quality [\[3\]](#).

4. Security as a Continuous Concern: DevSecOps

Integrating security considerations early in the development process will become the standard, evolving into a more mature and automated form of DevSecOps. Security will be an intrinsic part of the workflow, not only be a final hurdle. AI will be instrumental in this principle, “shifting left”,

continuously scanning code for vulnerabilities, analyzing dependencies for potential threats, and ensuring compliance with security policies from the initial design phase through to production[\[4\]](#).